

Organización de Datos

Organizaciones Indexadas

Material de estudio — Ingeniería Informática

Autores: Carlos Alberto Rico · Valeria A. Di Croce

Grupo de Investigación en Datos e Información (GIDI)

Contenidos del material

1. Modelo Entidad-Relación
2. Organizaciones Indexadas y Multilistas
3. Archivos Indexados
4. Índices: Denso vs. No Denso
5. Tipos de Índices (Primario, Secundario, Multinivel)
6. Índice de Agrupamiento
7. Archivos Invertidos
8. Comparativa de Organizaciones

✓ **Buena práctica:** Este documento es un resumen explicativo en profundidad de la unidad temática sobre Organizaciones Indexadas. Está diseñado para ser autónomo: puede estudiarse sin necesidad de acceder a la presentación original. Los conceptos se presentan en orden lógico, conectando cada idea con la siguiente.

1. El Modelo Entidad-Relación (ER)

■ 1.1 ¿Qué es una entidad?

El Modelo Entidad-Relación (ER) es una herramienta conceptual que permite representar la estructura de la información de un dominio del mundo real antes de llevarla a una base de datos. El concepto central es el de **entidad**: cualquier cosa del mundo real que sea *única y distinguible* de otras cosas, incluso si pertenece al mismo tipo. Una persona se diferencia de otra aunque sean gemelas; un automóvil se distingue por su número de chasis, motor o patente; una casa, por su dirección exacta.

Las entidades pueden ser de dos tipos. Las **concretas** tienen existencia física y tangible (una persona, un animal, una casa), mientras que las **abstractas** existen como conceptos o hechos sin materialidad física (un puesto de trabajo, una asignatura universitaria, un punto matemático). Esta distinción es importante porque condiciona cómo se identifican y acceden los datos.

■ 1.2 Atributos y claves

Cada entidad se describe mediante sus **atributos**: características que la definen. Por ejemplo, la entidad Persona puede tener los atributos Nombre, Apellido, Género y Fecha de Nacimiento. La entidad Libro tiene Título, Autor y Editorial. Entre los atributos existe una categoría especial: los **identificadores** (o atributos clave), que permiten distinguir unívocamente cada instancia de la entidad. El DNI identifica a una persona de forma única; el ISBN identifica a un libro. En el diagrama ER, los atributos clave se representan con elipse subrayada. También pueden existir **identificadores externos**, propios de entidades débiles (ver más abajo).

■ 1.3 Relaciones y cardinalidad

Una **relación** es el vínculo semántico que conecta dos o más entidades. En el diagrama ER se representa mediante un rombo con nombre propio. Por ejemplo: *Cliente* → *realiza* → *Pedido*, o *Pedido* → *se compone de* → *Artículo*. Cada relación tiene asociada una **cardinalidad** que indica cuántas instancias de una entidad pueden relacionarse con cuántas instancias de la otra.

Cardinalidad	Descripción	Ejemplo
1 : 1 (Uno a Uno)	Un A se relaciona con exactamente un B y viceversa	Empleado — tiene — Escritorio
1 : N (Uno a Muchos)	Un A puede relacionarse con varios B, pero cada B con un solo A	Cliente — realiza — Pedidos
N : 1 (Muchos a Uno)	Varios A se relacionan con un solo B	Pedidos — pertenecen — Cliente
N : M (Muchos a Muchos)	Varios A pueden relacionarse con varios B y viceversa	Pedido — se compone de — Artículos

■ 1.4 Entidades débiles

Una **entidad débil** es aquella que no puede identificarse por sí sola sin la ayuda de otra entidad (llamada entidad fuerte). El ejemplo clásico es Apartamento respecto de Edificio: el número de apartamento 3A no es único si no se especifica en qué edificio está. Por eso, la clave primaria de Apartamento se construye combinando el identificador del edificio y el número de apartamento: **id_edificio + nro_apartamento**. En el diagrama ER, la entidad débil se dibuja con doble rectángulo y su relación con doble rombo.

■ **Nota:** La diferencia entre entidad fuerte y débil no es de importancia semántica, sino de dependencia identificadora. Un Apartamento existe físicamente, pero su identidad depende del Edificio que lo contiene.

2. Organizaciones Indexadas: Multilistas

■ 2.1 Concepto de multilista

Una **multilista** es una estructura de datos que extiende la idea de lista enlazada permitiendo que cada nodo pertenezca simultáneamente a más de una lista. En el contexto de archivos y bases de datos, esto significa que un registro puede estar vinculado a otras entidades o grupos mediante múltiples punteros, sin necesidad de duplicar la información. La multilista es, en esencia, la implementación en memoria o en disco del Modelo Entidad-Relación: captura las relaciones N:M entre entidades de forma eficiente.

Existen tres variantes básicas de listas enlazadas que sirven como fundamento. Las **listas enlazadas simples** dividen la información en dos áreas (datos e índices) y cada nodo contiene un puntero al siguiente. Las **listas doblemente enlazadas** añaden un segundo puntero al nodo anterior, lo que facilita recorridos bidireccionales. Por último, las **listas circulares** conectan el último nodo de vuelta al primero, formando un anillo que permite recorrer la estructura indefinidamente.

■ 2.2 Implementación de una multilista

Para implementar una multilista que capture una relación entre dos entidades A y B, se necesitan al menos cuatro estructuras: una para los nodos de la entidad A, otra para los nodos de la entidad B, una tercera para los nodos de relación (que conectan A con B), y una estructura envolvente (NodoMultilista) que agrupa todo. Cada nodo de la entidad A contiene un puntero al primer nodo de relación que lo involucra. Cada nodo de relación contiene dos punteros: uno al siguiente nodo de relación para esa A, y otro al siguiente nodo de relación para esa B. Así se navega por todos los B asociados a un A determinado, y viceversa.

■ **Nota:** Ejemplo: si A = Estudiante y B = Asignatura, los nodos de relación conectan cada estudiante con las asignaturas que cursa. Para encontrar todas las asignaturas de Alex, se recorre la lista de relaciones que parten de su nodo. Para encontrar todos los alumnos de CS202, se recorre la lista de relaciones que llegan a ese nodo.

■ 2.3 Multilistas en archivos

Cuando la multilista se lleva a disco (archivo), los 'punteros' de memoria se reemplazan por **direcciones físicas o lógicas** dentro del archivo (como números de registro relativo, NRR). Los componentes son: los **nodos/registros** del archivo principal, las **relaciones** representadas mediante punteros físicos o lógicos entre registros, y los **archivos auxiliares (índices)** que mantienen esas relaciones y facilitan las búsquedas.

El ejemplo práctico del material es un sistema de biblioteca. Cada registro de libro contiene punteros a la lista de autores y a la lista de categorías. Los archivos auxiliares de autores y categorías mantienen, a su vez, punteros hacia atrás a los libros correspondientes. Al buscar todos los libros de un autor, se accede al archivo de autores, se localiza el registro de ese autor y se recorre su lista de libros sin duplicar ningún

dato.

■ 2.4 Ventajas e inconvenientes

Criterio	Detalle
✓ Eficiencia	Sin duplicación de datos: cada dato existe una sola vez
✓ Flexibilidad	Permite representar relaciones N:M y múltiples categorías simultáneamente
✓ Escalabilidad	Apropiada para grandes volúmenes con muchas relaciones cruzadas
✗ Complejidad	La implementación es más compleja que listas simples; hay que rastrear varios punteros
✗ Costo de acceso	Puede requerir buscar en varios archivos o índices para una sola consulta
■ Limitación	Es recomendable sólo cuando la búsqueda se hace sobre un único atributo a la vez

■ **Advertencia:** La complejidad de las multilistas crece rápidamente cuando hay muchas relaciones simultáneas. Si se necesita buscar combinando varios campos al mismo tiempo, conviene evaluar otras organizaciones como los archivos invertidos.

3. Archivos Indexados

■ 3.1 Concepto de archivo indexado

Un **archivo indexado** es un archivo cuyos registros pueden accederse de forma eficiente mediante una **clave de búsqueda**, gracias a la existencia de una estructura auxiliar llamada índice. La idea central es separar la 'dirección' del dato de la información misma. En lugar de recorrer todo el archivo secuencialmente para encontrar un registro, primero se consulta el índice (que es pequeño y puede estar ordenado o en memoria), se obtiene la dirección del registro deseado, y luego se hace un único acceso directo al dato. Este proceso de dos pasos —*búsqueda en índice + acceso al dato*— es la esencia de toda organización indexada.

Físicamente, el índice puede verse como un **vector (tabla) de registros**, cada uno con dos campos: la clave de organización y un puntero al dato. El puntero puede ser una dirección absoluta en disco o un Número de Registro Relativo (NRR). El archivo suele implementarse en dos ficheros separados: el fichero de índice y el fichero de datos.

■ 3.2 Área Principal y Área de Índices

El archivo indexado tiene dos zonas lógicas. El **Área Principal** almacena los registros con los datos reales, tal como fueron ingresados. El **Área de Índices** mantiene, para cada registro, su clave y un puntero a su posición en el área principal. Agregar un registro (anexar) genera un nuevo registro en ambas áreas. Eliminar un registro no requiere borrar el dato: basta con marcar su entrada en el índice con un puntero nulo. Insertar puede hacerse anexando o reutilizando una posición eliminada anteriormente.

■ **Nota:** Ejemplo numérico: si el archivo de datos tiene registros con claves {1, 5, 20, 10, 4, 21, 22} en esas posiciones físicas, el área de índices tiene esas mismas claves con sus respectivos punteros. Para buscar el registro con clave 10, se recorre (o se hace búsqueda binaria en) el índice, se obtiene el puntero a la posición 3 del área principal, y se accede directamente.

■ 3.3 Organización Secuencial Indexada

En la **organización secuencial indexada**, el archivo de datos está ordenado físicamente por la clave de indexación, y el índice se construye sobre esa secuencia. Esto permite dos modos de acceso complementarios: el **acceso secuencial** (recorrer el archivo en orden de clave, ideal para reportes o procesamiento masivo) y el **acceso directo** (buscar un registro particular mediante el índice, sin recorrer todo el archivo). Se pueden tener tantos índices como se necesite, variando la clave de indexación.

El índice puede organizarse a su vez de tres formas típicas: secuencial simple, multinivel (índice sobre índice) o en forma de árbol (árbol B o B+). Cada una ofrece un compromiso diferente entre velocidad de búsqueda y costo de mantenimiento.

✓ **Buena práctica:** Ventaja clave de la indexación: el índice es mucho más pequeño que el archivo de datos, por lo que cabe más fácilmente en memoria. Esto aumenta el factor de bloque y mejora drásticamente la eficiencia de búsqueda, incluso con búsqueda binaria.

4. Índice Denso vs. Índice No Denso (Escaso)

■ 4.1 Índice denso

Un **índice denso** (o total) contiene una entrada por cada registro del archivo de datos. Si el archivo tiene 1.000 registros, el índice tiene 1.000 entradas. La búsqueda es directa: se localiza la clave en el índice y se sigue el puntero. El problema es que el índice puede volverse tan grande que pierde su ventaja de tamaño. Si el índice no cabe en memoria, deja de ser eficiente.

■ 4.2 Índice no denso (escaso)

La solución al problema de tamaño es el **índice no denso** (o escaso): en lugar de indexar cada registro, se indexa sólo un *subconjunto de claves*. Típicamente, se indexa la clave del primer registro de cada bloque físico del archivo (el 'ancla' del bloque). Para buscar un registro, se localiza en el índice la entrada cuya clave es menor o igual a la buscada, se va al bloque indicado, y se busca secuencialmente dentro de ese bloque.

El índice no denso aprovecha el hecho de que el archivo de datos está ordenado físicamente: si sé que la clave 13654109 corresponde al inicio del bloque 3, entonces sé que el registro con clave 13654109 está en ese bloque o en los inmediatamente siguientes. Esto reduce el tamaño del índice de manera drástica manteniendo una búsqueda eficiente.

Característica	Índice Denso	Índice No Denso
Entradas en el índice	Una por cada registro	Una por cada bloque (o ancla)
Tamaño del índice	Grande	Mucho más pequeño
Búsqueda	Directa, sin leer el dato	Requiere búsqueda dentro del bloque
Requisito del archivo	No necesita estar ordenado	Debe estar ordenado por la clave
Uso típico	Índices secundarios	Índices primarios (ancla de bloque)

✓ **Buena práctica:** La elección entre denso y no denso depende del tamaño del archivo y de si éste está ordenado físicamente. Para archivos muy grandes, el índice no denso es casi siempre preferible como índice primario, reservando el denso para índices secundarios.

5. Tipos de Índices: Primario, Secundario y Multinivel

■ 5.1 Índice primario

El **índice primario** se construye sobre la **clave primaria** del archivo de datos, es decir, el campo que determina el orden físico de los registros. Como el archivo está ordenado por esa clave, el índice primario puede ser no denso: basta con almacenar la clave del ancla de cada bloque. Cada entrada del índice apunta al bloque donde comienza el conjunto de registros que tienen esa clave o mayor. La búsqueda es muy eficiente: se hace búsqueda binaria en el índice y luego se accede directamente al bloque correspondiente.

■ 5.2 Índice de agrupamiento

El **índice de agrupamiento** (o clustering index) se utiliza cuando el archivo de datos está ordenado por un campo que *no es clave*, es decir, un campo con valores repetidos. Por ejemplo, si el archivo de empleados está ordenado por número de departamento (numDpto), varios registros comparten el mismo valor de numDpto. El índice de agrupamiento tiene una entrada por cada valor distinto del campo de agrupamiento, y cada entrada apunta al bloque donde comienzan los registros de ese grupo. Esto permite recuperar todos los empleados de un departamento de forma muy eficiente.

Una variante importante es el **índice de agrupamiento con bloques individuales**: en lugar de que los registros de un mismo grupo queden en bloques contiguos del archivo principal, se reservan bloques separados para cada grupo, encadenados mediante punteros de bloque. Esto facilita las inserciones sin necesidad de reorganizar el archivo completo.

■ 5.3 Índice secundario

El **índice secundario** se construye sobre cualquier campo que no sea la clave de ordenación física del archivo. Pueden ser claves candidatas u otros campos no clave. A diferencia del primario, el índice secundario es siempre **denso** (una entrada por cada registro), porque el archivo de datos no está ordenado por ese campo: no se puede asumir que registros con valores similares están en bloques cercanos. Pueden existir múltiples accesos al mismo bloque del archivo de datos desde diferentes entradas del índice.

Cuando el campo de indexación secundaria *no es clave* (puede tener valores repetidos), aparece un problema adicional: varias entradas del índice podrían apuntar al mismo valor. La solución es usar un **nivel de indirección**: el índice tiene valores únicos que apuntan a estructuras intermedias (listas de punteros a bloques), y estas listas apuntan a los registros reales. Así el índice mantiene entradas de longitud fija con valores únicos.

■ 5.4 Índice multinivel

Si el índice primario es aún demasiado grande para buscar eficientemente, la solución es construir un **índice sobre el índice**: un índice multinivel. El nivel base (primer nivel) es el índice original que apunta a los datos. El segundo nivel es un índice no denso sobre el primer nivel, que apunta a bloques del primer nivel. Pueden añadirse más niveles hasta que el nivel superior quepa cómodamente en memoria. La búsqueda desciende nivel a nivel, y en cada nivel se hace búsqueda binaria o directa.

El árbol B+ es la implementación generalizada y autobalanceada de esta idea: los nodos internos actúan como índices de nivel superior, y las hojas contienen los datos o punteros a ellos, encadenadas entre sí para facilitar el recorrido secuencial.

✓ **Buena práctica:** Regla general: índice primario → no denso; índice secundario → denso; si el índice todavía es grande → agregar niveles (multinivel o árbol B+).

6. Archivos Invertidos

■ 6.1 Motivación y concepto

Cuando un archivo tiene varios índices secundarios, surgen problemas prácticos importantes. Los índices secundarios contienen valores repetidos (salvo que sean sobre campos clave). Cualquier actualización, inserción o eliminación de datos obliga a **reactualizar todos los índices** secundarios afectados, lo que puede ser muy costoso. Además, el acceso a través de un índice secundario invierte el proceso habitual: en lugar de ir del dato al índice, vamos del índice al dato mediante la búsqueda secundaria.

Un **archivo invertido** es un archivo organizado a partir de **índices secundarios basados en la clave primaria**. Es 'invertido' porque invierte la relación entre datos e índices: en lugar de almacenar los datos ordenados por la clave primaria y el índice como auxiliar, aquí los índices secundarios son protagonistas. Existen dos tipos: **totalmente invertidos** (se crean índices secundarios sobre todos los campos) y **parcialmente invertidos** (sólo sobre algunos campos seleccionados).

■ 6.2 Referencia en índices secundarios: NRR vs. clave primaria

Una decisión de diseño central en los archivos invertidos es a qué apuntan las entradas del índice secundario. Hay dos alternativas:

Criterio	Apuntar al NRR (posición física)	Apuntar a la clave primaria
Volatilidad	Alta (si el dato se mueve, hay que actualizar TODOS los índices)	Baja (la clave primaria no cambia aunque el dato se mueva)
Consulta	Rápida: 1 acceso directo al dato	Más lenta: 2 accesos (índice secundario → clave primaria → dato)
Mantenimiento	Costoso: reorganización implica actualizar todos los índices	Simple: no hay que tocar los índices secundarios al reorganizar
Independencia	Baja: depende de la ubicación física	Alta: independencia lógica de la posición del dato

■ 6.3 Implementación: listas invertidas

La implementación más común agrupa las referencias en **listas invertidas**. En lugar de tener en el índice secundario una entrada por cada registro (lo que generaría muchas filas con el mismo valor, por ejemplo 'BEETHOVEN' repetido cuatro veces), se almacena cada valor único *una sola vez* con un puntero a una lista separada que contiene todas las claves primarias (o NRRs) de los registros que tienen ese valor.

Ejemplo del material: en un archivo de grabaciones musicales, el campo 'compositor' puede tener valores como BEETHOVEN, COREA, DVORAK, etc. El índice secundario por compositor tiene una sola entrada para BEETHOVEN, que apunta a la lista {ANG3795, DG139201, DG18807, RCA2626}, las cuatro claves primarias de las grabaciones de Beethoven. La lista invertida separada permite que el índice tenga

entradas de longitud fija y facilita las operaciones de mantenimiento.

✓ **Buena práctica:** Ventajas de las listas invertidas: gestión más eficiente en operaciones de mantenimiento (inserción y eliminación de referencias) y menor ocupación de memoria que tener las listas embebidas en el propio índice.

■ **Nota:** Los archivos invertidos permiten responder a consultas complejas (combinando varios campos) directamente sobre los índices, sin necesidad de acceder al archivo de datos. Por ejemplo: 'grabaciones de Beethoven en sello DG' se resuelve intersectando la lista de Beethoven con la lista del sello DG.

■ **Advertencia:** Sistemas muy dinámicos (muchas actualizaciones) → conviene apuntar a clave primaria. Sistemas orientados a lectura (muchas consultas, pocas modificaciones) → conviene apuntar al NRR.

7. Organización B+ y Árbol Indexado

■ 7.1 Árbol B vs. Árbol B+

El **árbol B+** es una variante del árbol B optimizada para sistemas de archivos. Mientras que en un árbol B los datos pueden estar en cualquier nodo (internos y hojas), en un árbol B+ toda la información de datos queda *exclusivamente en las hojas*. Los nodos internos actúan como 'separadores' del índice, conteniendo copias de claves (que pueden repetirse). Las hojas están enlazadas entre sí en una lista secuencial, lo que hace que el recorrido en orden sea extremadamente eficiente.

Característica	Árbol B	Árbol B+
Datos	En nodos internos y hojas	Solo en las hojas
Claves	Únicas en todo el árbol	Pueden repetirse (copias en nodos internos)
Búsqueda	Puede terminar en cualquier nivel	Siempre termina en las hojas
Hojas	No necesariamente enlazadas	Enlazadas en lista secuencial
Recorrido secuencial	Menos eficiente	Muy eficiente (recorre hojas)
Altura	Mayor	Menor (mejor rendimiento)

■ 7.2 Organización B+ como índice separado

La organización B+ puede usarse como estructura del índice de un archivo secuencial-indexado. En este esquema, el **fichero de datos** está ordenado secuencialmente (conjunto secuencia), y el **árbol B+** constituye el índice que permite la búsqueda directa. Los nodos hoja del árbol B+ apuntan a bloques del fichero de datos. Para el acceso directo se desciende por el árbol; para el acceso secuencial se recorren las hojas encadenadas del árbol, que ya están en orden.

Cuando el archivo de datos crece y algún bloque se llena, los nuevos registros van a una **zona de desbordamiento** (overflow). Cada registro que 'desborda' queda en esa zona enlazado desde el bloque original, y el árbol se actualiza con el nuevo puntero. Periódicamente, el archivo necesita ser **reestructurado** para consolidar los bloques de overflow y mantener las prestaciones de búsqueda.

✓ **Buena práctica:** En general, es mejor mantener el índice separado del archivo de datos (dos ficheros distintos). Esto permite mayor eficiencia en la organización secuencial, mayor seguridad ante errores y reconstrucción simple del índice a partir del archivo de datos si fuera necesario.

8. Comparativa de Organizaciones de Archivos

■ 8.1 Los tres enfoques principales

La organización de archivos puede abordarse desde tres enfoques fundamentales, cada uno con su propia filosofía de acceso. La **organización secuencial** almacena los datos en orden y el acceso se realiza recorriendo el archivo linealmente: es simple y muy eficiente para procesar todos los registros, pero lenta para búsquedas puntuales. La **organización indexada** (típicamente con árbol B+) combina un índice de búsqueda con el archivo de datos, ofreciendo un equilibrio entre acceso puntual rápido y recorrido secuencial eficiente. La **organización hash** aplica una función matemática a la clave para calcular directamente la posición del registro, logrando el acceso más rápido posible para búsquedas exactas, pero sin soporte para rangos u orden.

■ 8.2 Tabla comparativa

Criterio	Secuencial	Indexada (B+)	Hash
Búsqueda puntual	Lenta (lineal)	Rápida — $O(\log n)$	Muy rápida — $O(1)$
Recorrido secuencial	Excelente	Muy bueno (hojas del B+)	Malo (sin orden)
Inserciones	Costosas (reorganizar)	Balanceadas (splits en B+)	Muy rápidas
Eliminaciones	Costosas	Controladas (merge en B+)	Simples
Reorganización	Frecuente y costosa	Automática en B+	Rehash necesario
Uso de espacio	Bajo (sin estructuras)	Medio (requiere índices)	Variable (puede crecer)
Flexibilidad	Baja	Alta	Baja

Criterio	Secuencial	Indexada (B+)	Hash
Búsqueda puntual	Lenta (lineal)	Rápida — $O(\log n)$	Muy rápida — $O(1)$
Recorrido secuencial	Excelente	Muy bueno (hojas del B+)	Malo (sin orden)
Inserciones	Costosas (reorganizar)	Balanceadas (splits en B+)	Muy rápidas
Eliminaciones	Costosas	Controladas (merge en B+)	Simples
Reorganización	Frecuente y costosa	Automática en B+	Rehash necesario
Uso de espacio	Bajo (sin estructuras)	Medio (requiere índices)	Variable (puede crecer)
Flexibilidad	Baja	Alta	Baja

■ 8.3 ¿Cuándo conviene cada organización?

Organización	Cuándo usarla	Ejemplo típico
--------------	---------------	----------------

Secuencial	Procesos batch; lecturas completas del archivo; pocos cambios en los datos	Listados, reportes, históricos
Indexada (B+)	Muchas búsquedas por clave; necesidad de orden + acceso rápido; consultas mixtas (puntuales y por rango)	Bases de datos generales
Hash	Acceso directo por clave exacta; muchas consultas puntuales; el orden no importa	Tablas de acceso rápido, cachés

■ 8.4 Regla mnemotécnica final

Una forma sencilla de recordar la elección correcta: **Si quiero recorrer** → **secuencial** (simple, bajo costo de espacio, ideal para leer todo). **Si quiero equilibrio** → **indexado** (busco rápido, leo en orden, mantengo bien). **Si quiero velocidad pura** → **hash** (acceso en $O(1)$, sin importar el orden). En términos de complejidad general: secuencial es *simple pero lento*, indexado es *equilibrio*, hash es *rápido pero rígido*.

■ **Advertencia:** Ninguna organización es universalmente superior. La elección depende del patrón de uso: frecuencia de inserciones y eliminaciones, proporción de búsquedas puntuales vs. secuenciales, tamaño del archivo y disponibilidad de memoria para los índices.